Mise au point d'une surcouche floue sur un Système de Gestion de Bases de Données

Garreau Pierre-Jean Guitton Denis Le Quéré Damien Massiot Paul

16 décembre 2009

Table des matières

1 Introduction : rappels sur les BD floues **3**
1.1 FQUERY . . . 3
1.2 SQL-FLOU . . . 4

2 Fonctionnement de notre surcouche floue **5**
2.1 Concept et Structure de la base de données utilisée . . . 5
2.1.1 Création des types . . . 5
2.1.2 Gestion des prédicats flous . . . 5
2.1.3 Utilisation de données floues . . . 7
2.2 Requètes floues . . . 7
2.2.1 Exemple de fonctionnement . . . 7
2.2.2 Transformation de la Requete . . . 8
2.3 Operations floues . . . 9
2.3.1 Sélection floue . . . 9
2.3.2 Jointures floues . . . 12
2.3.3 Opérateurs logiques usuels sur des données floues . . . 12
2.3.4 Opérateur flou sur des données exactes : la fonction environ égal . . . 12

3 Présentation de la maquette **13**
3.1 Diagramme de cas d'utilisation . . . 13
3.2 Un premier aperçu de l'interface graphique . . . 13

4 Organisation de la suite du projet **17**
4.1 Choix techniques . . . 17
4.2 Planification et répartition des tâches . . . 17

Chapitre 1

Introduction : rappels sur les BD floues

Les bases de données floues répondent à plusieurs besoins dans le domaine des bases de données. D'abord le besoin d'obtenir des résultats moins tranchés qu'en SQL classique avec l'algèbre de Bool sur des données précises (FQUERY) ; puis le besoin de stocker des données imprécises et de les manipuler, voire d'utiliser des prédicats flous sur ces données (SQL-FLOU).
Pour répondre à ces besoins, les bases de données floues s'appuient sur la logique floue, plus particulièrement sur les sous-ensembles flous et les prédicats flous.

1.1 FQUERY

Sous-ensemble flou : Un sous-ensemble flou A de E est défini par une fonction d'appartenance (appelée *prédicat flou*) f_A qui associe à chaque élément x de E le degré d'appartenance $f_A(x)$ avec lequel il appartient à A (compris entre 0 et 1). Exemple :
$EnsAge = \{12, 20, 78, 23, 45, 60\}$; $f_{estVieux} : EnsAge \rightarrow [0, 1]$

- $f_{estVieux} : 20 \rightarrow 0$
- $f_{estVieux} : 78 \rightarrow 1$
- $f_{estVieux} : 45 \rightarrow 0.25$
- $f_{estVieux} : 50 \rightarrow 0.5$

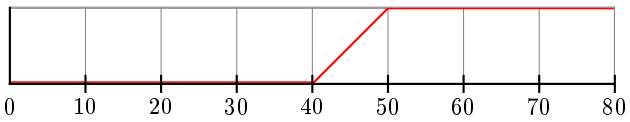


FIG. 1.1 – Représentation graphique du prédicat $f_{estVieux}$

Le principe de FQUERY est de permettre d'effectuer des requêtes floues c'est à dire des requêtes exprimées en SQL, dont la partie condition est étendue pour traiter les prédicats flous définis. On considère des tables SQL classiques contenant des données précises. Le résultat d'une requête floues est une relation floue pour laquelle chaque tuple a un degré d'appartenance appartenant à [0,1] et décrivant le succès de la requête.

Exemple d'une requête floue simple

```
SELECT * FROM Personnes WHERE estVieux(Age);
```

Nom	Prénom	Age
Durand	Jean	48
Dupond	Pierre	21
Dubois	Paul	16
Thomas	Jacques	55
Jagger	Tutu	42

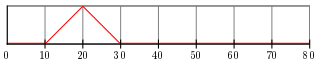
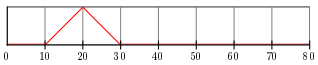
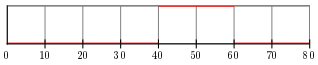
$\Longrightarrow$

Nom	Prénom	Age	estVieux(Age)
Durand	Jean	48	0.8
Thomas	Jacques	55	1
Jagger	Tutu	42	0.2

TAB. 1.1 – Traitement de la requête floue

1.2 SQL-FLOU

L'idée de base de FSQL est comme pour FQUERY d'utiliser les prédicats flous pour effectuer des requêtes floues sur des données précises, mais il s'agit en plus de pourvoir stocker des données imprécises et d'être capable de leur appliquer des prédicats flous.
Les moyens utilisés sont les prédicats flous pour stocker les données imprécises, et des mesures issues de la théories des possibilités pour comparer des prédicats flous entre eux (mesures de la possibilité et de la nécessité). On utilise ces mesures pour fournir les résultats de requêtes floues sur des données floues.

Nom	Prénom	Age
Durand	Jean	
Dupond	Pierre	
Dubois	Paul	
Thomas	Jacques	
Jagger	Tutu	

TAB. 1.2 – Exemple d'une table comportant des données imprécises

Notre travail est d'imaginer des solutions permettant d'intégerer ces deux aspects FQUERY et FSQL à un système de gestion de bases de données existant.

Chapitre 2

Fonctionnement de notre surcouche floue

2.1 Concept et Structure de la base de données utilisée

2.1.1 Création des types

Nous étions confronté à un problème : comment déterminer à quel type de données s'applique un prédicat ? On comprend que rattacher chaque prédicat à un type usuel est insuffisant. Par exemple, le prédicat **vieux** s'applique sur des entiers mais seulement sur ceux correspondant au concept d'*age*. Pour cela, nous avons créé une table des types dans laquelle on référence les noms des types conceptuels (par exemple l'age) et les types de base de SQL (ici Integer lorsqu'on connaît précisement l'age et Integer-flou si on souhaite stocker des données imprécises).

Il est donc possible de créer deux genres de types : les types correspondant à des données exactes et les types correspondant à des données floues. Les types flous permettent de stocker les données floues correspondantes, on pourra leur appliquer des opérations floues. Les types exacts stockeront des données précises, on pourra aussi leur appliquer des opérateurs flous.

2.1.2 Gestion des prédicats flous

Comme nous l'avons vu dans l'introduction, les opérations floues et les données floues sont stockées sous la forme de prédicats flous. Ainsi nous avons choisi de stocker tous les prédicats d'un type concret (*integer*, *integer-flou*, *réel-flou*...) dans une seule table (avec par exemple la convention que les prédicats des données floues ne seront pas nommés). On aura donc une table pour chaque type concret - car les prédicats sont stockés de la même façon pour chaque type concret.

Comme nous établissons une correspondance entre un prédicat flou et le type de son argument, nous aurons dans la table le champ *type* précisant le type de l'argument du prédicat. Bien entendu il faut aussi mémoriser les valeurs numériques de chaque prédicat. Quatre valeurs au maximum sont nécessaires pour représenter les fonctions floues envisagées (on stocke les abscisses des points de rupture). Pour ne pas avoir à stocker les valeurs des ordonnées nous avons choisi un système de code pour chaque "patron de fonction floue". On stockera également ce code qui, combiné aux valeurs numériques nous permettra de retrouver le prédicat flou. On peut imaginer par exemple de stocker les valeurs et le code d'un prédicat dans un même tableau (fixe) de cinq éléments.

Convention de codage des différentes formes de prédicats flous



FIG. 2.1 – Code 0 : Valeur Exacte

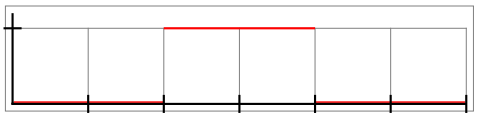


FIG. 2.2 – Code 1 : Intervalle plein



FIG. 2.3 – Code 2 : Interval vide

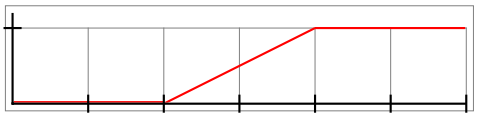


FIG. 2.4 – Code 3 : Côte

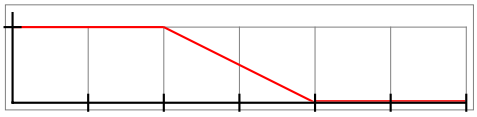


FIG. 2.5 – Code 4 : Pente

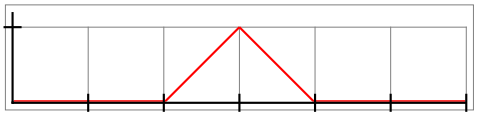


FIG. 2.6 – Code 5 : Pyramide

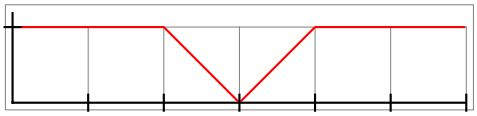


FIG. 2.7 – Code 6 : Gouffre

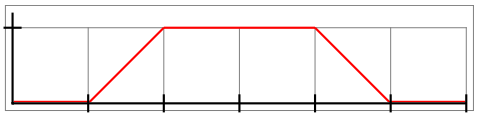


FIG. 2.8 – Code 7 : Trapèze



FIG. 2.9 – Code 8 : Trapèze inversé

2.1.3 Utilisation de données floues

Chaque entrée d'un type flou dans une table doit stocker une données floues. Pour cela, données floue de la table est un pointeur vers un tuple d'une des tables de prédicats. Lors de l'ajout d'un tuple contenant une donnée floue, l'utilisateur peut :

- réutiliser une donnée floue existante (pour éviter les redondances)
- créer une nouvelle donnée floue, et dans ce cas il lui proposé de réutiliser sont prédicat flou comme un prédicat de requête

2.2 Requètes floues

2.2.1 Exemple de fonctionnement

Prenons comme exemple une base de données possédant, entre autres, une table T_VOITURE. Cette table stocke des données sur des voitures d'occasion avec une estimation de leur prix.

Modèle	Prix	Age	Compteur
BX	→ 01	15	350000
ZX	→ 02	13	175000
407	→ 03	1	2000
Clio	→ 04	5	150000

FIG. 2.10 – T_VOITURE

Le champ *Modèle* est une chaîne de caractères, nous n'allons pas lui appliquer de prédicat flou. Les champs *Age* et *Compteur* sont des données précises de type entier, on crée un type de prédicat pour pouvoir leur appliquer un prédicat flou lors d'une requête SQL. Enfin, le champ *Prix* contient

un pointeur vers le prédicat représentant chaque valeur. Pour ce dernier, lors de la création du type de prédicat, on précise que l'on souhaite travailler sur des entiers flous. La table T_TYPE stoke les informations nécessaires aux types de prédicat.

NomType	TypeSource
TAge	Integer
TPrix	IntegerFlou
TCompteur	Integer

FIG. 2.11 – T_TYPE

Dans la table des prédicats sont stockés les données imprécises de l'exemple mais aussi des prédicats que nous utiliserons avec les requêtes SQL. Les prédicats peuvent être nommés, cela n'est pas indispensable, en particulier pour les prédicats représentant une donnée floue. Le type du prédicat est requis. Pour finir, un tableau de cinq valeurs précise le patron du prédicat et les valeurs correspondantes.

NomPredicat	TypePredicat	Valeurs
null	TPrix	`[[5],[1700],[2000],[2100],[]]`
null	TPrix	`[[7],[2700],[2800],[3200],[3300]]`
null	TPrix	`[[0],[15000],[],[],[]]`
null	TPrix	`[[7],[4600],[4800],[5000],[5200]]`
vieux	TAge	`[[3],[7],[10],[],[]]`
cher	TPrix	`[[3],[5000],[10000],[],[]]`
usé	TCompteur	`[[3],[150000],[200000],[],[]]`

FIG. 2.12 – T_PREDICAT

2.2.2 Transformation de la Requete

Premier exemple : appliquer un prédicat à une donnée précise. L'exemple suivant montre les différentes étapes de l'exécution d'une requete SQL floue. On souhaite obtenir les modèles de voiture agés. Le prédicat **vieux** est donc appliqué au champ *Age*. La première étape consiste à réécrire la requête afin de la transmettre au SGBD. Celui-ci nous renvoie le résultat de la requête. Pour terminer, le degré d'appartenance est calculé et retourné avec les données.

```
SELECT Modèle, Age          SELECT Modèle, Age
FROM T_VOITURE        →     FROM T_VOITURE
WHERE cher(Prix)            WHERE Prix > 5000
```

↓

Modèle	Age	Degré
BX	15	1
ZX	13	1

←

Modèle	Age
BX	15
ZX	13

FIG. 2.13 – Exemple d'application d'un prédicat

Deuxième exemple : appliquer un opérateur logique à deux prédicats. Si la requête SQL contient un opérateur logique sur deux prédicats, celle-ci est décomposée en deux sous-requêtes envoyées séparément au SGBD. Les deux tables résultantes seront alors fusionnées en une seule table qui sera renvoyée à l'utilisateur.

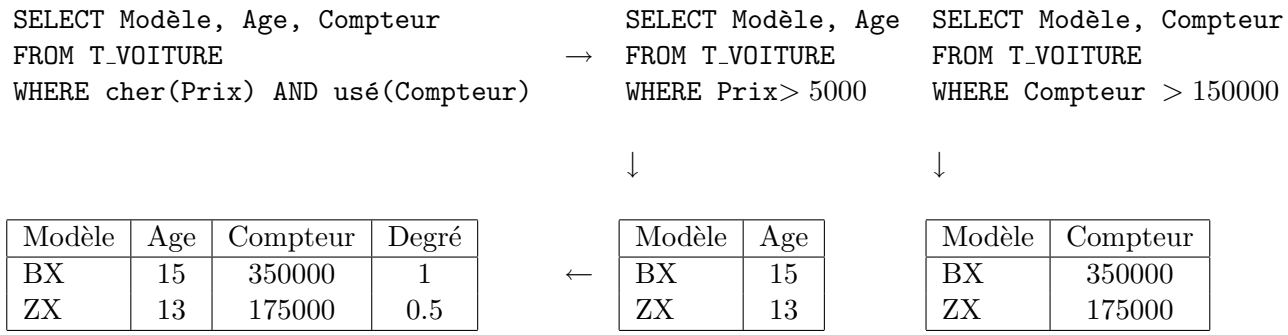


FIG. 2.14 – Exemple d'application d'un opérateur logique

2.3 Operations floues

2.3.1 Sélection floue

La sélection floue consiste à appliquer un prédicat flou construit par l'utilisation de l'opérateur unaire NOT et/ou des opérateurs binaires AND et OR sur les prédicats flous donnés.

La négation. Si f est la fonction d'appartenance définissant le prédicat flou **cher**, alors la négation de ce prédicat NOT(cher) est définit par 1-f.

La conjonction. La conjonction de deux prédicats flous A et B sera construite à partir des tuples retournés et par P1 et par P2. Un tuple retourné par un seul des deux prédicats ne sera pas présent dans le résultat final. Le degré d'appartenance des tuples valides aura pour valeur le minimum des degrés calculés pour les prédicats A et B.

La disjonction. La disjonction de deux prédicats flous A et B sera construite à partir des tuples retournés soit par A soit par B. Si un même tuple est retourné par les deux prédicats, le degré d'appartenance aura pour valeur le maximum des degrés calculés pour les prédicats A et B.

Opérations flous sur données exactes

Dans ce cas, le calcul est simple. On utilise le système de code et les éléments ti du tableau :

Si Code=0 : valeur exacte
- 0 si $(x \neq t1)$
- 1 si $(x = t1)$

Si Code=1 : interval plein
- 0 si $(x < t1)$ ou $(x > t2)$
- 1 si $(x > t1)$ ou $(x < t2)$

Si Code=2 : interval vide

- 0 si $(x > t1)$ ou $(x < t2)$
- 1 si $(x < t1)$ ou $(x > t2)$

Si Code=3 : côte

- 0 si $(x < t1)$
- 1 si $(x > t2)$
- sinon $\frac{x-t1}{t2-t1}$

Si Code=4 : pente

- 0 si $(x > t1)$
- 1 si $(x < t2)$
- sinon $\frac{t2-x}{t2-t1}$

Si Code=5 : pyramide

- 0 si $(x < t1)$ ou $(x > t3)$
- 1 si $(x = t2)$
- si $(x > t1)$ et $(x < t2)$ $\frac{x-t1}{t2-t1}$
- si $(x > t2)$ et $(x < t3)$ $\frac{t3-x}{t3-t2}$

Si Code=6 : pyramide inversé

- 0 si $(x = t2)$
- 1 si $(x < t1)$ ou $(x > t3)$
- si $(x > t1)$ et $(x < t2)$ $\frac{t2-x}{t2-t1}$
- si $(x > t2)$ et $(x < t3)$ $\frac{x-t2}{t3-t2}$

Si Code=7 : trapèze

- 0 si $(x < t1)$ ou $(x > t4)$
- 1 si $(x > t2)$ ou $(x < t3)$
- si $(x > t1)$ et $(x < t2)$ $\frac{x-t1}{t2-t1}$
- si $(x > t3)$ et $(x < t4)$ $\frac{t4-x}{t4-t3}$

Si Code=8 : trapèze inversé

- 0 si $(x > t2)$ ou $(x < t3)$
- 1 si $(x < t1)$ ou $(x > t4)$
- si $(x > t1)$ et $(x < t2)$ $\frac{t2-x}{t2-t1}$
- si $(x > t3)$ et $(x < t4)$ $\frac{x-t3}{t4-t3}$

Opérations flous sur données floues

Dans ce cas, on cherche à mesurer quelle est la probabilité que la donnée floue corresponde à l'opération floue. Il existe deux méthodes pour calculer ce résultat :

La mesure de possibilité : PS(a / b) = sup min (Fa(x),Fb(x))
La mesure de nécessité : NE(a / b) = inf max (1-Fa(x),Fb(x))

Il est relativement aisé pour un humain de comprendre le principe de ces deux mesures. Par exemple pour la mesure de possibilité, on cherche la borne supérieure du minimum des deux fonctions. Cependant pour qu'une machine puisse le calculer, il faut décomposer le graphe des deux fonctions afin d'obtenir des sous-graphes représentant les fonctions sans variation. Nous nous sommes rendus compte que ce problème revient à trouver des extremums. Or il est assez facile de travailler dessus grâce à notre solution qui stocke les extremums de nos prédicats flous. On obtient donc des domaines de définition en fonction des intervalles definis par nos extrenums sur lesquel nos fonctions sont monotones.

Ensuite, il faut distinguer 10 cas :
- F1 décroissante et F2 décroissante : retourner min(Valeur(F1,x1),Valeur(F2,x1))
- F1 décroissante et F2 croissante : retourner égales(F1,F2,x1,x2,seuil)
- F1 décroissante et F2 Stable à 1 : retourner Valeur(F1,x1)
- F1 croissante et F2 décroissante : retourner égales(F1,F2,x1,x2,seuil)
- F1 croissante et F2 croissante : retourner min(Valeur(F1,x2),Valeur(F2,x2))
- F1 croissante et F2 Stable à 1 : retourner Valeur(F1,x2)
- F1 Stable à 1 et F2 décroissante : retourner Valeur(F2,x1)
- F1 Stable à 1 et F2 croissante : retourner Valeur(F2,x2)
- F1 Stable à 1 et F2 Stable à 1 : retourner 1
- F1 Stable à 0 ou F2 Stable à 0 : retouner 0

fonction égales On sait que les deux fonctions se coupent en un point inconnu. Le but de l'algorithme basé sur la recherche dichotomique est de trouver l'abscisse de ce point. On coupe l'intervalle en deux puis on cherche dans laquelle des deux parties les fonctions se coupent. Soient x1 et x2 respectivement les bornes minimale et maximale de l'intervalle, on regarde si F1(x1)-F2(x1) et F1(x1+x2)-F2(x1+x2) ont le même signe. Si tel est le cas alors on peut en déduire que les fonctions ne se coupent pas sur cet intervalle. Dans le cas contraire, on appelle récursivement cette fonction sur l'intervalle qui nous intéresse jusqu'à s'approcher d'une différence entre les fonctions inférieure à un seuil prédéfini proche de zéro.

```
Fonction égales(F1,F2,x1,x2,seuil)
debut
    si (abs(Valeur(F1,x1)-Valeur(F2,x2))<seuil)
        alors retourner x1
        sinon si ((Valeur(F1,x1)-Valeur(F2,x1))>0)==
Valeur(F1,((x1-x2)/2))-Valeur(F2,((x1-x2)/2)))>0))
alors égales(F1,F2,x1,((x1-x2)/2),seuil)
  sinon égales(F1,F2,((x1-x2)/2,x2,seuil)
finsi
finsi
fin
```

Un fois tous les maximums locaux calculés on defini lequel est le plus grand. C'est notre valeur de possibilité.

Pour la mesure de nécessité, une méthode similaire est envisagée.

2.3.2 Jointures floues

Pour les jointures floues (jointures sur des types flous) un seul opérateur est disponible, l'opérateur à peu près égal ($\simeq$). Celui-ci permet de regrouper les tuples de la manière suivante : On calcule la possibilité comme montré plus haut. On joint les tuples si la valeur de possibilité multipliée par cent est supérieure au delta definie par l'utilisateur. On affiche la correspondance entre les données floues jointes lors de l'affichage de la table.

2.3.3 Opérateurs logiques usuels sur des données floues

Les opérateurs logiques usuels sur des données floues sont convertis en prédicats flous. Par exemple , la fonction $= Y$ devient le prédicat flou :

- 0 si $x \neq Y$
- 1 si $x = Y$

La fonction $< Y$ devenant :

- 0 si $x > Y$
- 1 si $x < Y$

2.3.4 Opérateur flou sur des données exactes : la fonction environ égal

La fonction environ égale ($\simeq$) est une fonction qui permet de faire des approximations sur des données exactes. Elle est définie comme suit :

- 0 si $\frac{x1-x2}{\frac{x1+x2}{2}} \times 100 < \Delta$
- 1 sinon

Le delta est une valeur prédéfinie par l'utilisateur au moment de la création de la requête. Elle représente le pourcentage de correspondance entre les valeurs x1 et x2.

Chapitre 3

Présentation de la maquette

3.1 Diagramme de cas d'utilisation

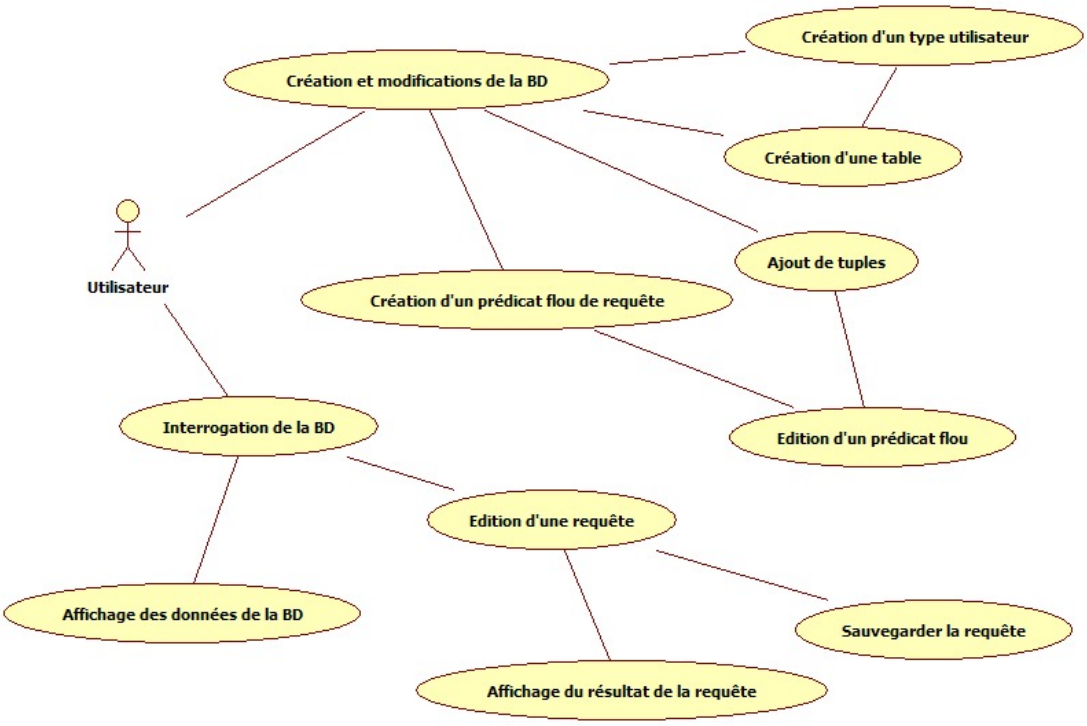


FIG. 3.1 – Diagramme des cas d'utilisation de l'application

3.2 Un premier aperçu de l'interface graphique

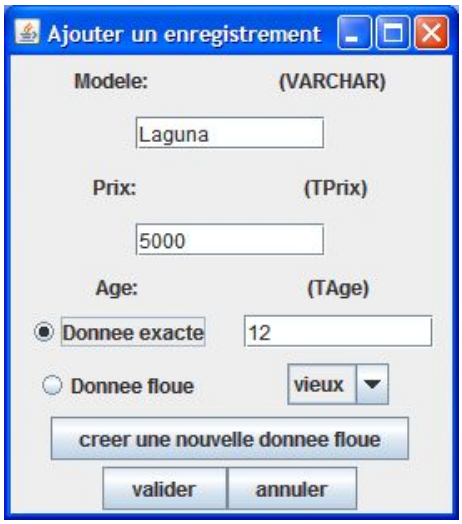


FIG. 3.2 – Ajout d'un enregistrement

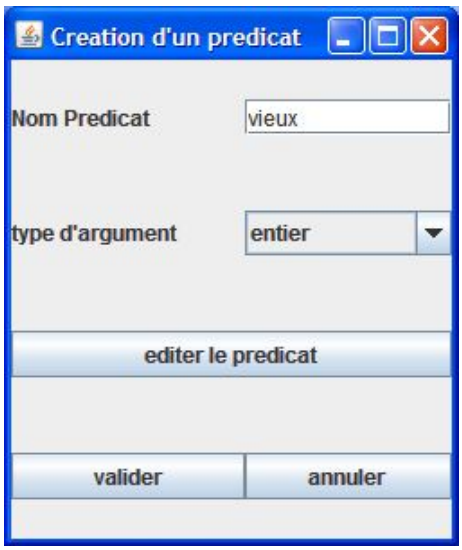


FIG. 3.3 – Création d'un prédicat de requête
s

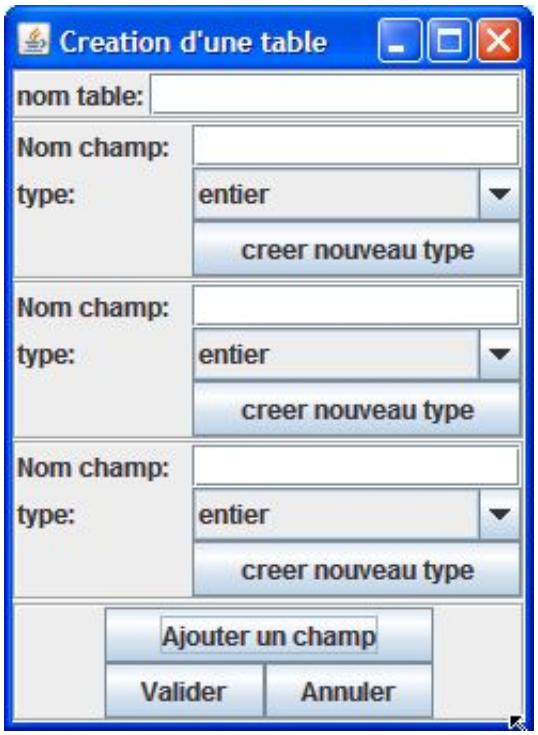


FIG. 3.4 – Création d'une table

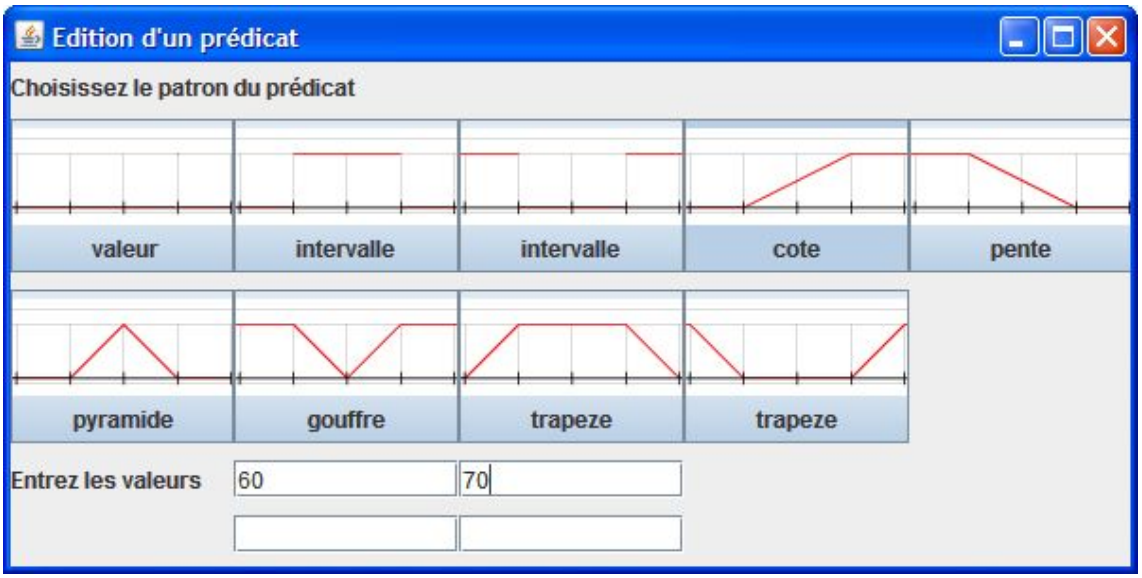


FIG. 3.5 – Editeur de prédicats flous

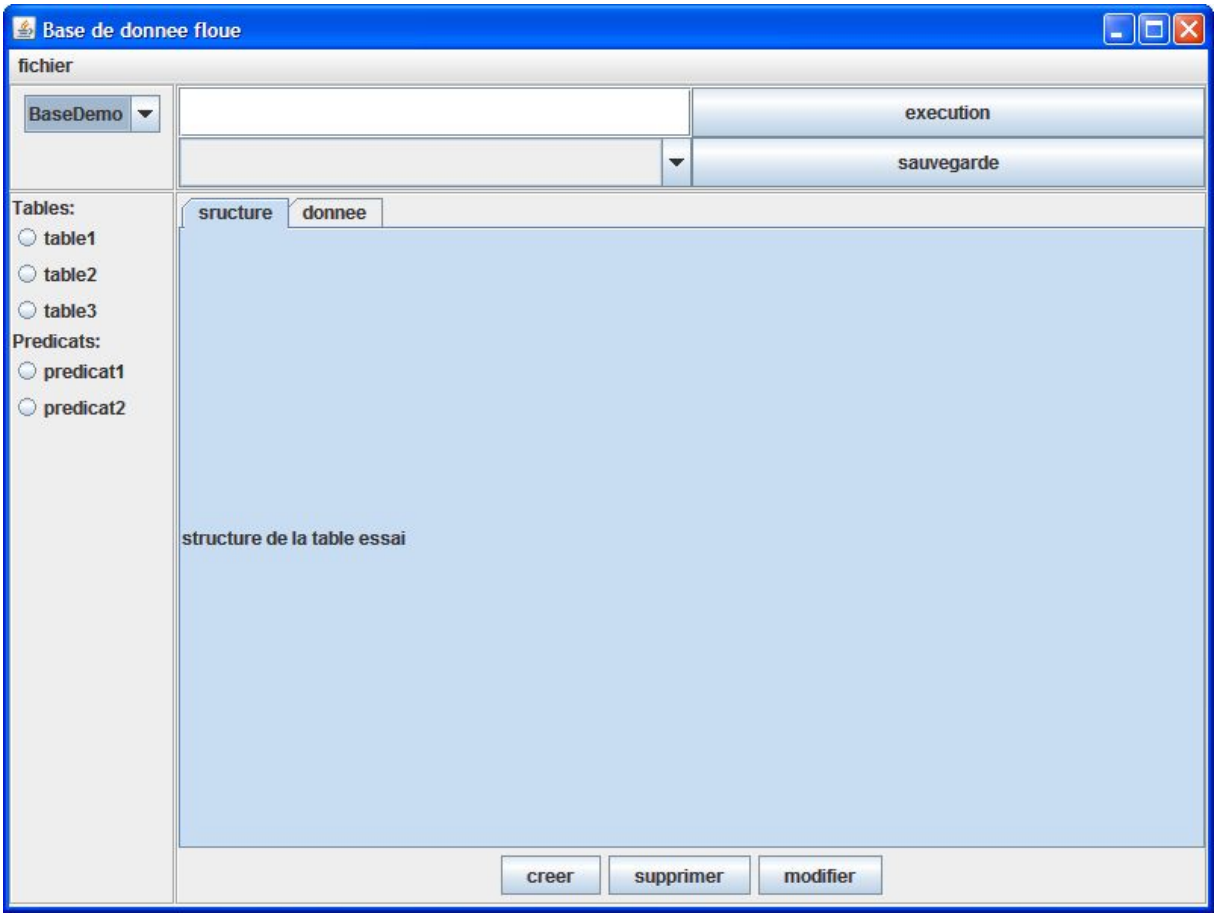


FIG. 3.6 – Interface principale

Chapitre 4

Organisation de la suite du projet

4.1 Choix techniques

- Langage de programmation : *Java*
- Base de données : *PostGre SQL*
- API d'accès aux bases de données : *JDBC*
- Dépot SVN sur *Google Code*

4.2 Planification et répartition des tâches

Planification

Décomposition en tâches :
- Réalisation de l'interface graphique
- Développement de l'interpreteur de requêtes
- Développement de la partie JDBC
- Développement de la partie calcul des tuples
- Rédaction du rapport + préparation de la soutenance

Tâches	*04/01*	*11/01*	*18/01*	*25/01*	*01/02*	*08/02*	*15/02*
Interface graphique	X	X	X	X	X		Soutenance
Interpreteur de requêtes	X	X	X	X	X		Soutenance
Partie JDBC	X						Soutenance
Partie calcul des tuples		X	X	X			Soutenance
Rapport + soutenance						X	Soutenance

Tab. 4.1 – Planification de développement du projet semaine par semaine (date du lundi)

Répartition des tâches

Nous envisageons de mettre une personne sur le développement de l'interface graphique tout le long du projet car c'est une tâche pour laquelle la compréhension du code est parfois fastidieuse. Les autres membres s'occuperont de la conception et du développement des solutions décrites dans cette première phase du projet en les remettant éventuellement en cause si des contradictions apparaissent.

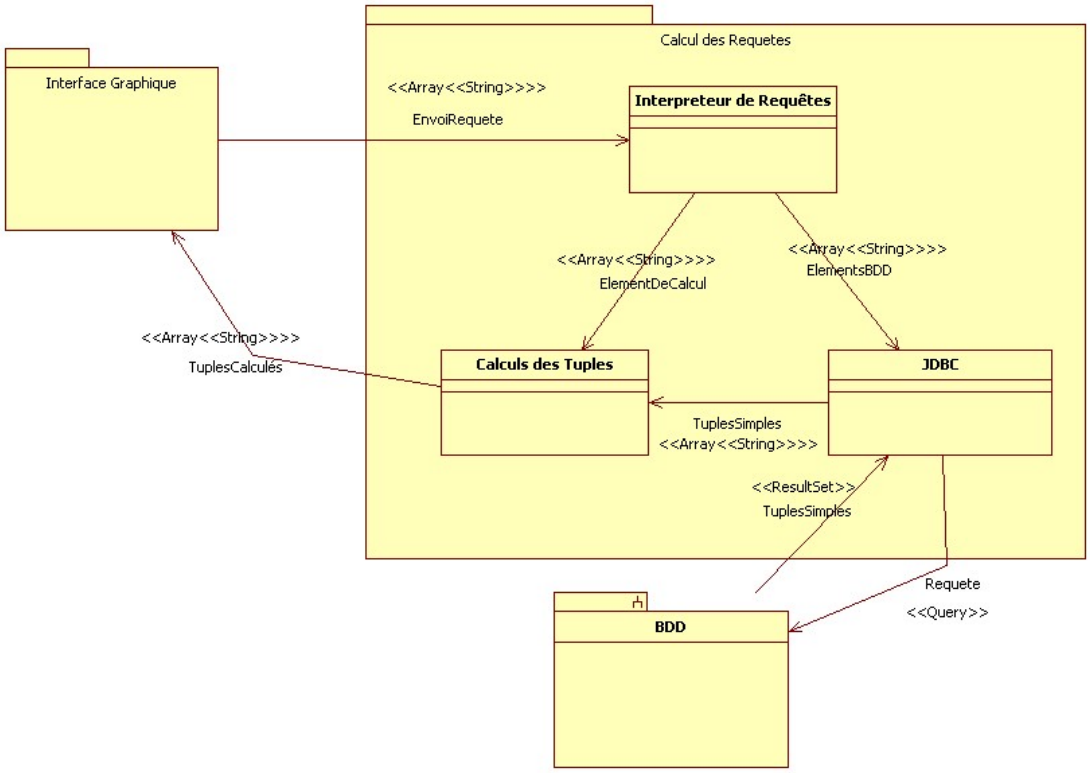


Fig. 4.1 – Diagramme de conception technique générale